

A
Project Report
On
“Android Application for Chatting”
Submitted to
Shivaji University, Kolhapur
In Partial fulfillment for the Degree of
Bachelor of Computer Application (Sem-VI)

By
Mr. Chandrakant Krishna Mane.
Mr. Mahesh Ganpati Mali.

Under the guidance of
Smt. Mrunal A. Thorat (BCA, MCA)

Through,



The Director,
Deccan Education Society's
Chintamanrao Institute of Management Development &
Research, Sangli

In the Year
2025 – 26



Deccan Education Society's
Chintamanrao Institute of Management
Development & Research, Sangli
Department of Computer Application

CERTIFICATE

This is to certify that the project report entitled “ **Android Application for Chatting**”. is record of project work, carried out in this college by **Mr. Chandrakant Krishna Mane, Mr. Mahesh Ganpati Mali**, in the partial fulfillment of award of **Bachelor of Computer Application (Sem– VI)** as laid down by Shivaji University, Kolhapur.

This report presents their sincere work carried out under my guidance.

Place: Sangli.

Date: 23/03/2026

Smt. Mrunal A. Thorat
Project Guide

Dr. Sweta Metha
Director

Examiner: 1) _____

2) _____

DECLARATION

To,
The Registrar,
Shivaji University,
Kolhapur.

We undersigned, **Mr. Chandrakant Krishna Mane, Mr. Mahesh Ganpati Mali**, here by to declare that, the project report entitled“ Android Application for Chatting”. submitted by us under the guidance of **Smt. Mrunal A. Thorat** is our original work. We have not reproduced any information about from any report submitted to Shivaji University, Kolhapur this year or any of the previous years.

We undersigned that any such coping is liable to the punishment in away that University Authorities deem fit.

Place: Sangli.

Date: 23/03/2026

Exam Seat No.	Name of Member	Signature
	Mr. Chandrakant Krishna Mane	
	Mr. Mahesh Ganpati Mali	

ACKNOWLEDGEMENT

We express our sincere gratitude and heartfelt appreciation for the opportunity to develop and present this project, “ **Android Application for Chatting**”. The journey of building this project has been a valuable learning experience, and we are thankful to all those who contributed to its success.

First and foremost, we extend our deepest gratitude to our guide **Smt. Mrunal A. Thorat** for her unwavering support, insightful guidance, and encouragement throughout the project. Her expertise and feedback were instrumental in shaping our work.

Finally, we acknowledge and appreciate all the online platforms, tools, and resources that contributed to our research and development process.

Place: Sangli.

Date: 23/03/2026

Exam Seat No.	Name of Member
	Mr. Chandrakant Krishna Mane
	Mr. Mahesh Ganpati Mali

INDEX

Chapter No.	Name Of Topic	Page No.
1	Introduction to Project 1.1 Introduction to Project Theme 1.2 Working of System 1.3 Need & Scope of Proposed System	1 2-3 4
2	Proposed System 2.1 Objectives of the system 2.2 Software Requirement Specification	5 6-7
3	System Diagrams 3.1 Context Level diagram 3.2 DFD 3.3 ERD 3.4 System requirements • Hardware requirements • Software requirements	8 9-11 12 13-14
4	System Design 4.1 Database Design (Data Dictionary) 4.2 Input Design 4.3 Output Design	15-17 18-41 42-46
5	System Implementation 5.1 User Manual	47-51
6	System Maintenance 6.1 Conclusion 6.2 Further Enhancement	52 53
7	Bibliography	54

Chapter 1

Introduction to Project

Android Application for Chatting

1.1 Introduction to Project Theme:

In today's digital era, mobile communication and instant messaging have become essential for personal and professional interaction. With the widespread use of Android smartphones, users expect fast, reliable, and interactive communication platforms. The proposed project, Vibe Chat Application, is developed using Java for the application logic and XML for designing the user interface. This combination allows creating a robust, responsive, and user-friendly mobile application.+-

The main theme of this project is to provide real-time text messaging between users in a secure and efficient manner. Unlike traditional messaging methods such as SMS or email, this application delivers messages instantly, ensuring smooth communication. The application also maintains a history of all conversations, which users can access at any time.

The project follows the client-server architecture, where the Android client interacts with the server to send and receive messages. User information, login credentials, and chat messages are securely stored in a database, enabling reliable message delivery and data consistency. XML is used to design the user interface, making it simple, visually appealing, and responsive on different screen sizes, while Java handles all the functional logic including message handling, database interaction, and user authentication.

Security and privacy are crucial aspects of this project. The system uses user authentication to ensure that only registered users can access the application. Each message is linked with sender and receiver details and timestamped for proper tracking.

This project not only helps in understanding Android app development using Java and XML, but also gives practical exposure to database management, networking concepts, and real-time application design. The Vibe Chat Application is designed to be scalable, secure, and efficient, with the potential for future enhancements like group chats, multimedia sharing, and notifications.

Overall, the project theme emphasizes real-time communication, mobile app development skills, security, and user-friendly interface, providing a solid foundation for both academic learning and industry-level software development.

1.2 Working of the System

1. Splash Screen

- Displays a welcome screen when the app is launched.
- Usually shows the app logo and loading animation while the app initializes resources in the background.

2. Sign Up / Registration

- New users provide details like username, email/mobile, and password.
- Data is validated and stored in the database.
- Ensures unique accounts and secure storage of credentials.

3. Login

- Registered users enter credentials to access the app.
- System verifies credentials with the database.
- Only authenticated users can access chat features.

4. Profile Management

- Users can view and update personal information such as name, profile picture, and status.
- Changes are stored in the database and reflected to other users in real-time.

5. Chat / Messaging

- Users select a contact and type messages.
- Messages are sent to the server, stored in the database, and delivered to the receiver.
- Real-time message delivery is provided if the receiver is online; otherwise, messages are delivered when they log in.

Android Application for Chatting

6. Voice Message

- Users can record and send voice messages directly in the chat.
- The recorded audio file is uploaded to the server and stored securely.
- The receiver can play the voice message instantly within the chat interface.

7. Video Call

- Users can initiate a video call with any contact.
- The system establishes a peer-to-peer connection or via server relay.
- Audio and video streams are transmitted in real time.

8. Camera / Media Capture

- Users can capture photos or videos using the device camera.
- Media files can be sent directly within the chat.
- Captured media is uploaded to the server and shared securely with the receiver.

9. Settings

- Users can customize application preferences such as notifications, privacy options, and theme.
- Users can manage account settings, change password, and update profile details.
- All changes are saved and applied instantly within the application.

10. Logout

- Users can safely log out of the app.
- User session is terminated to prevent unauthorized access.
- App redirects the user back to the login screen.

1.2 Need & Scope of proposed System:

Need of the proposed system

In today's digital world, users need fast and real-time communication. Traditional methods like email and SMS are slow and not suitable for instant interaction. The proposed Vibe Chat Application provides quick message delivery, making communication more efficient and effective.

The system is also designed to be user-friendly and secure. It ensures data privacy through authentication and stores chat history for future use. Overall, it improves communication by providing a simple, safe, and reliable platform.

Scope of the Proposed System -

The Vibe Chat Application allows users to register and log in securely. It supports real-time one-to-one messaging and stores user data and chat history in a database for easy access. The system works using an internet connection and can be used on compatible devices.

However, the current version includes only basic features like login, messaging, and logout. Advanced features such as group chat and file sharing are not included but can be added in future updates.

Chapter 2

Proposed System

2.1 Objectives of the System:

The primary objective of the Vibe Chat Application is to design and develop a secure, efficient, and user-friendly real-time messaging system similar to WhatsApp. The specific objectives of the system are as follows:

1. The system enables instant messaging between users with minimal delay using real-time communication technologies, making conversations fast and efficient.
2. The system provides secure registration and login functionality to protect user data and ensure privacy through proper authentication methods.
3. Users can easily share images, videos, documents, and other media files within chat conversations, making communication more interactive.
4. The system supports both individual and group chats, allowing users to communicate flexibly with one person or multiple users.
5. The application has a simple, easy-to-use, and responsive interface suitable for all users.
6. All chat messages are securely stored in a database, enabling users to access and review their chat history anytime.
7. The system is capable of handling multiple users at the same time while maintaining smooth and efficient performance.
8. The system provides instant notifications for new messages, helping users stay updated and improving overall engagement.

2.2 Software Requirement Specification:

1. Introduction

The Software Requirement Specification (SRS) document describes the functional and non-functional requirements of the **Vibe Chat Application**. This document serves as a guideline for system developers and users to understand the system's features, behavior, and constraints.

1. Functional Requirements

1. User Registration –

- Allows new users to create an account and start using the application easily

2. User Login & Logout –

- Enables secure access to the system and proper session management

3. One-to-One Messaging –

- Allows direct and private communication between two users in real-time

4. Group Chat Creation –

- Enables users to create groups and communicate with multiple users together

5. Real-Time Message Sending & Receiving –

- Ensures messages are delivered instantly without delay

6. Image, Video & Document Sharing –

- Allows users to share media files and important documents

7. Message History Storage –

- Stores all previous conversations for future access and reference

8. Notification for New Messages –

- Notifies users instantly when a new message is received

Android Application for Chatting

2. Non-Functional Requirements

1. Fast Message Delivery –

- Ensures messages are delivered quickly with minimal delay for smooth communication

2. Secure User Authentication –

- Protects user data with safe login and authentication mechanisms

3. High System Reliability –

- Ensures the application works consistently without crashes or failures

4. User-Friendly Interface –

- Provides a simple and easy-to-use interface for all users

5. Scalable System Performance –

- Supports increasing users and data without affecting performance

6. Limited Advanced Features –

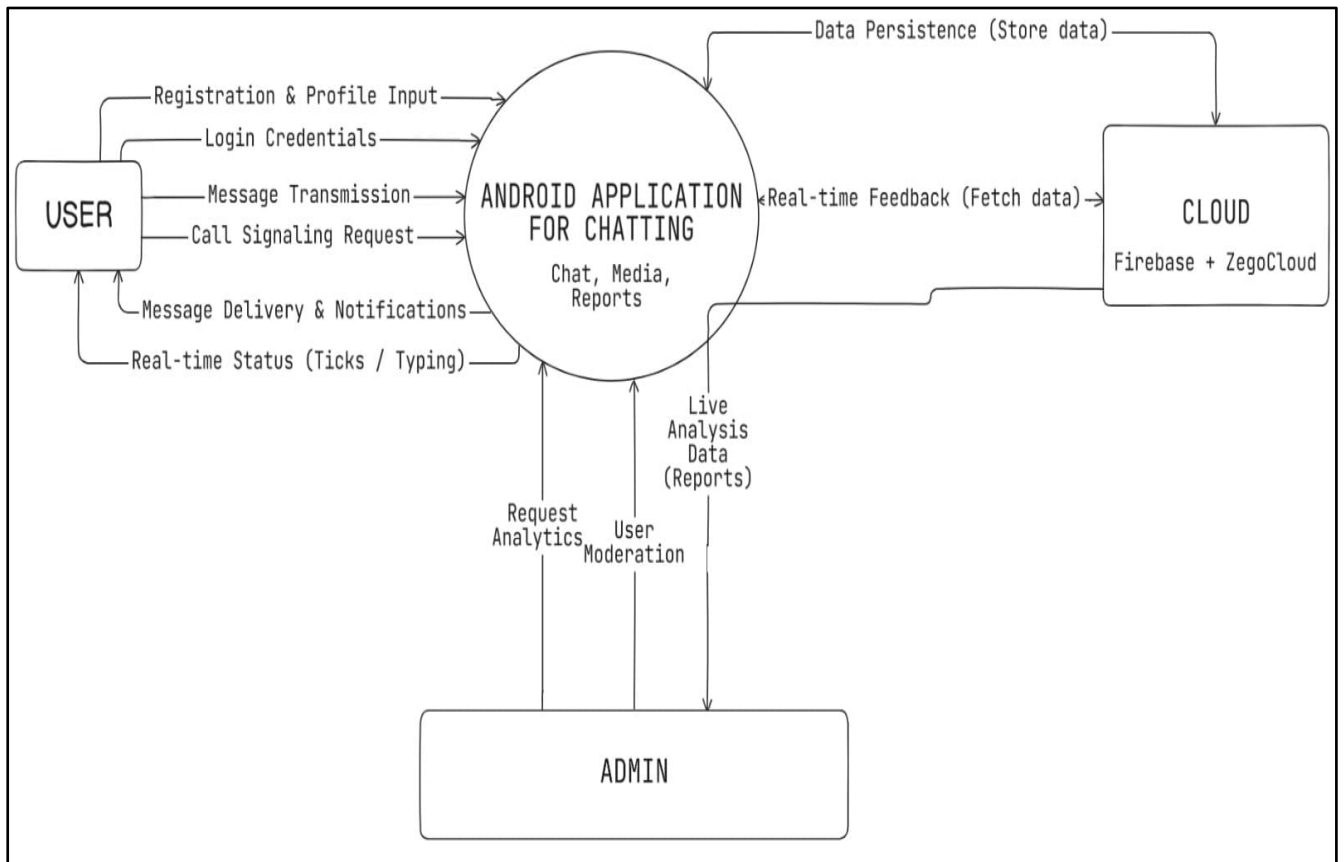
- Focuses on essential features to keep the system simple and efficient

Chapter 3

System Diagrams

Android Application for Chatting

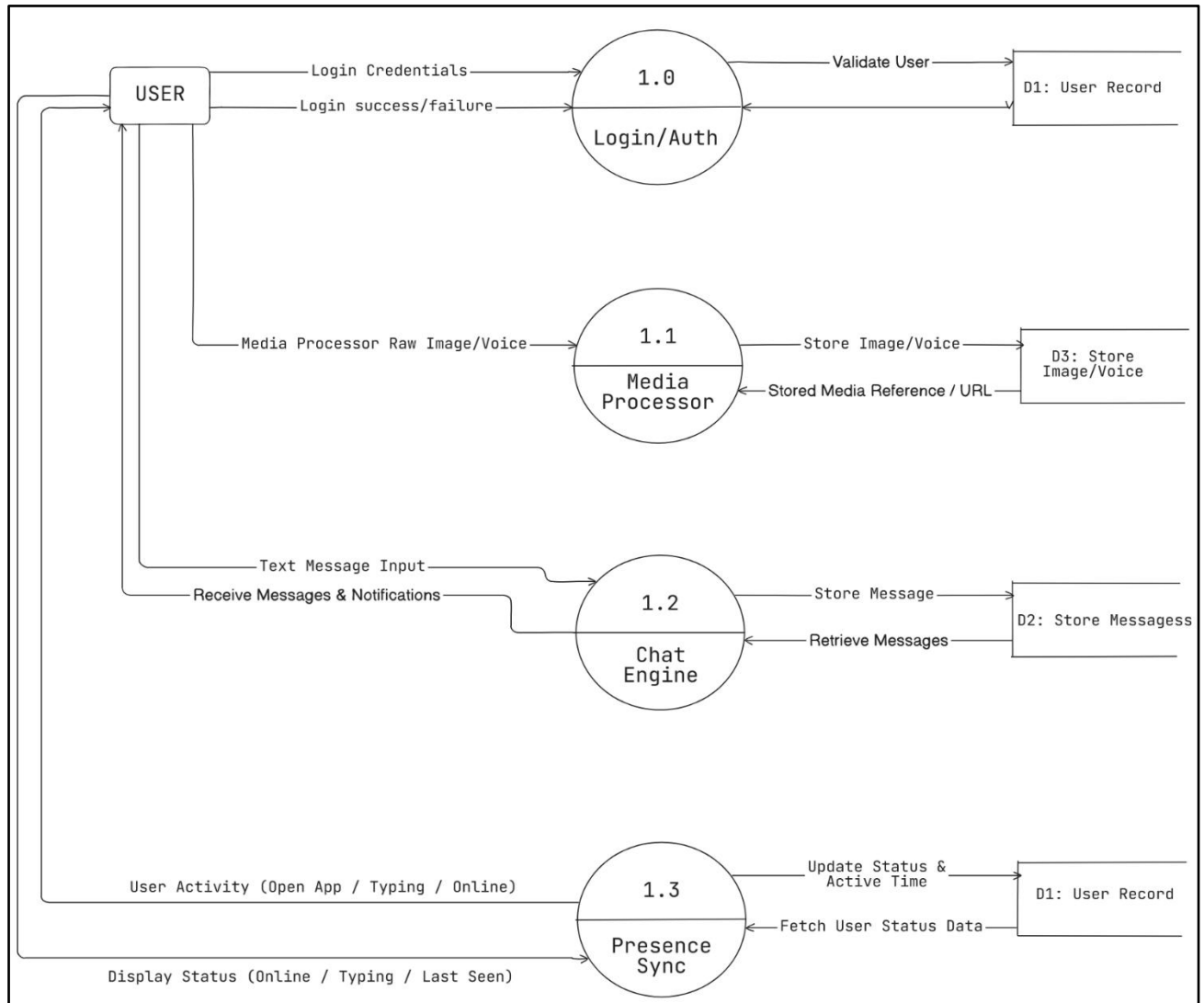
3.1 Context Level diagram



Android Application for Chatting

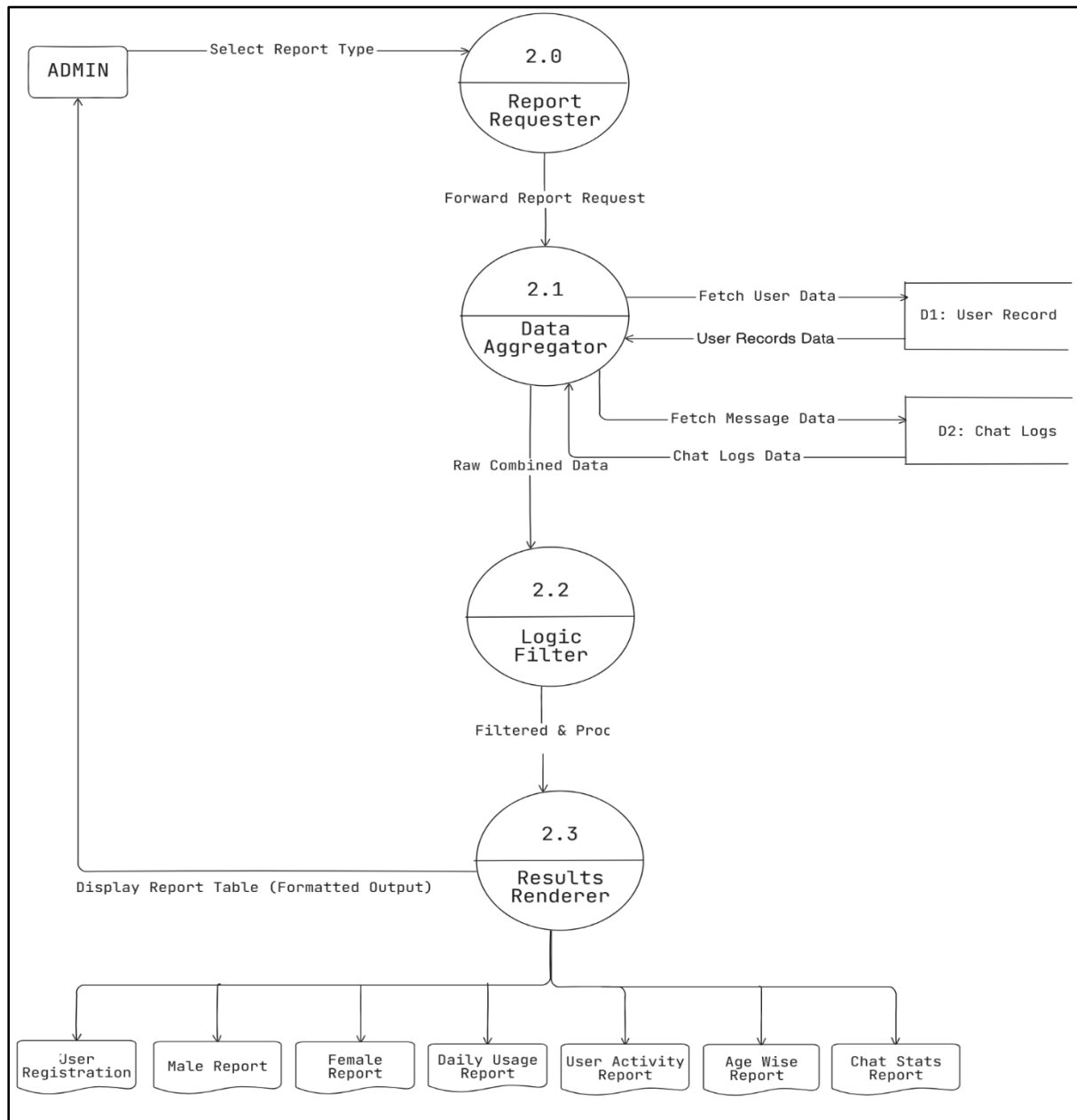
3.2 Data Flow Diagram -

1. User communication DFD -



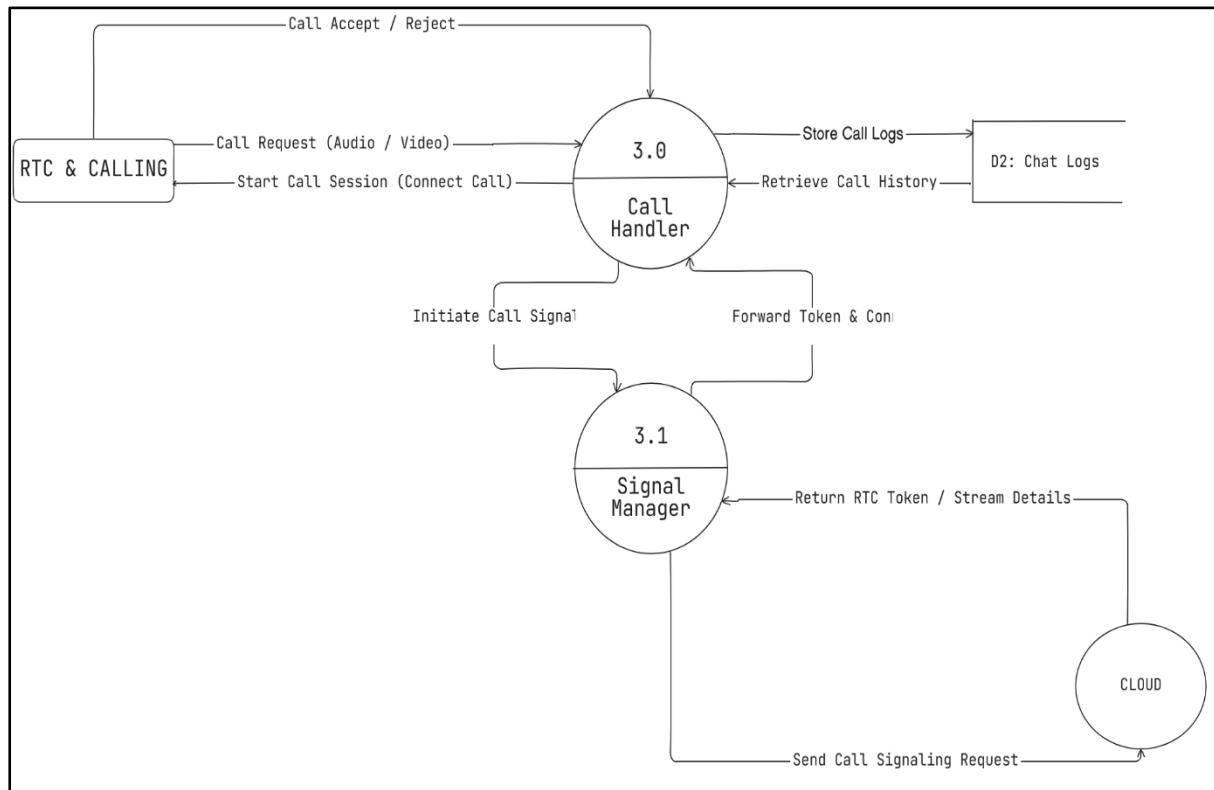
Android Application for Chatting

2 – Admin and Reporting DFD



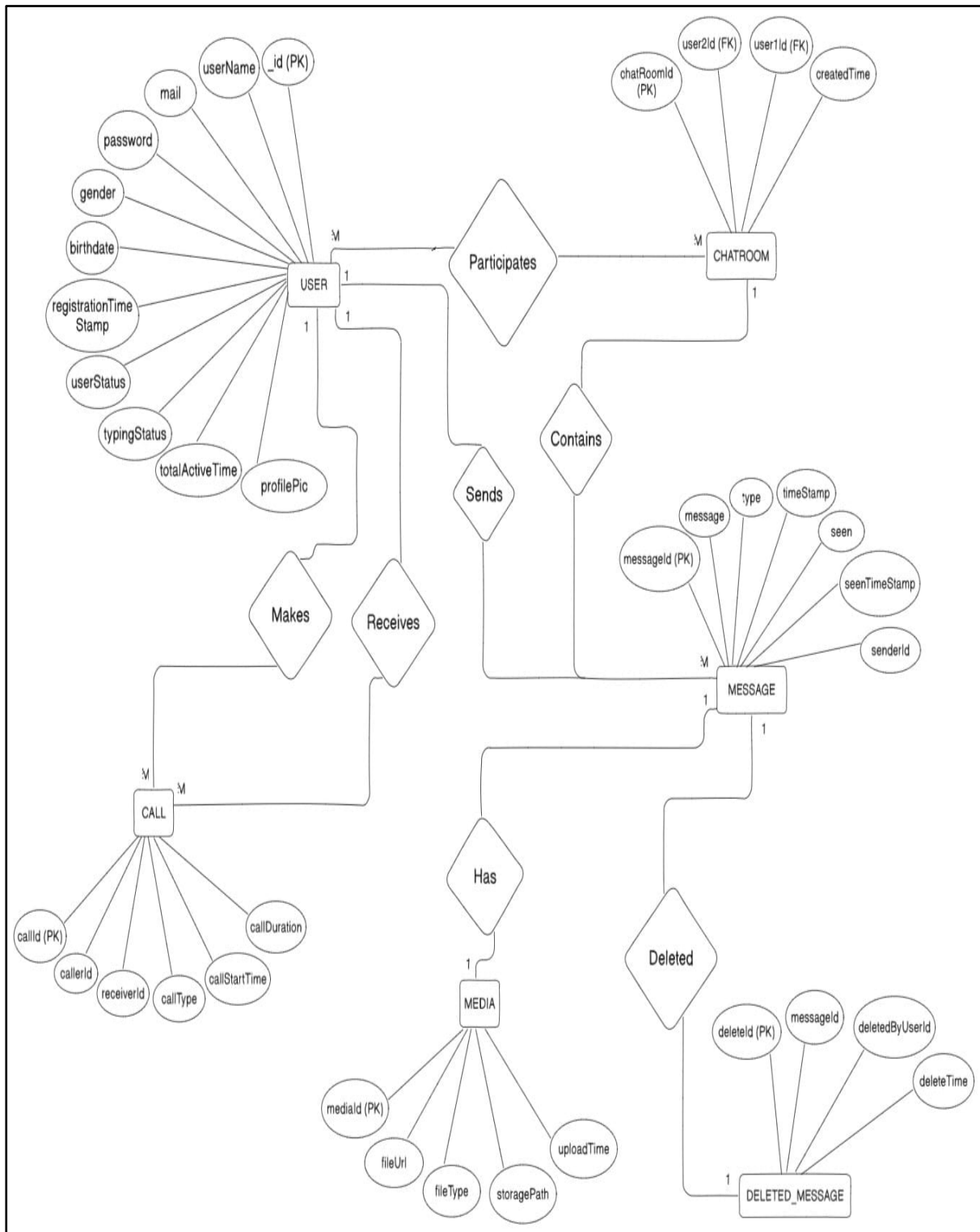
Android Application for Chatting

3. RTC & Calling DFD



Android Application for Chatting

3.2 ERD Diagram



3.4 . System Requirements

1. Hardware Requirements

A. Development Environment (Developer Side)

Processor: i5 (Min), i7 / Ryzen 7 (Recommended)

RAM: 8 GB (Min), 16 GB (Recommended)

Storage: 100 GB free (SSD preferred)

Internet: High-speed stable connection

OS: Windows 10/11, macOS 10.14+, Linux

B. End-User Environment (User Side)

Device: Android Phone/Tablet

Camera & Mic: Required (calls, media)

Connectivity: 4G/5G or Wi-Fi

2. Software Requirements

A. Development Tools

IDE: Android Studio (Iguana / Jellyfish)

Language: Java (JDK 8/11)

Database: Firebase Realtime DB

Storage: Firebase Storage

Auth: Firebase Authentication

Calling SDK: ZegoCloud UIKit

API: Volley (FCM requests)

Android Application for Chatting

B. Libraries / Dependencies

Image: Picasso

UI: Circle Image View, Card View, Recycler View, SDP/SSP

Media: Media Recorder, Media Player

Notifications: Firebase Cloud Messaging (FCM)

3. Operational Requirements

Min Android: 7.0 (API 24)

Target Android: 14+ (API 34+)

Permissions

INTERNET

CAMERA

RECORD_AUDIO

READ / WRITE STORAGE (below Android 10)

POST_NOTIFICATIONS (Android 13+)

SYSTEM_ALERT_WINDOW (call overlay)

4. System Architecture

Backend: Firebase (Serverless)

Realtime: Firebase (Web Sockets)

RTC: Zego Cloud

Notifications: FCM (HTTP / V1)

Chapter 4

System Design

Android Application for Chatting

4.1 Database Design (Data Dictionary)

1. User Table-

Field Name	Data Type	Description
User Id (PK)	String	Unique ID of user
User Name	String	Name of user
email	String	Email of user
password	String	User password
gender	String	Gender of user
Birth date	String	Date of birth
Profile Pic	String	Profile image URL
User Status	String	Online / Offline / Last Seen
Typing Status	String	Typing indicator
Total Active Time	Long	Total active time of user
Registration Time Stamp	Long	Account creation time
Created At	Date	Record creation time
Updated At	Date	Record update time

2. Chat Room Table -

Field Name	Data Type	Description
Chat Room Id (PK)	String	Unique chat room ID
user1Id (FK)	String	First user ID
user2Id (FK)	String	Second user ID
Created Time	Long	Chat created timestamp

Android Application for Chatting

3. Message Table -

Field Name	Data Type	Description
Message Id (PK)	String	Unique message ID
chatroom Id (FK)	String	Chat room reference
Sender Id (FK)	String	Sender user ID
message	String	Message content
type	String	text / image / voice
Time Stamp	Long	Message sent time
seen	Boolean	Message seen status
Seen Time Stamp	Long	Message read time

4. Media Table -

Field Name	Data Type	Description
Media Id (PK)	String	Unique media ID
Message Id (FK)	String	Related message
File Url	String	File download URL
fileType	String	image / voice
Storage Path	String	Firebase storage path
Upload Time	Long	Upload timestamp

5. Call Table -

Field Name	Data Type	Description
Call Id (PK)	String	Unique call ID
Caller Id (FK)	String	Caller user ID
Receiver Id (FK)	String	Receiver user ID
Call Type	String	audio / video
Call Start Time	Long	Call start time
Call Duration	Long	Duration in seconds

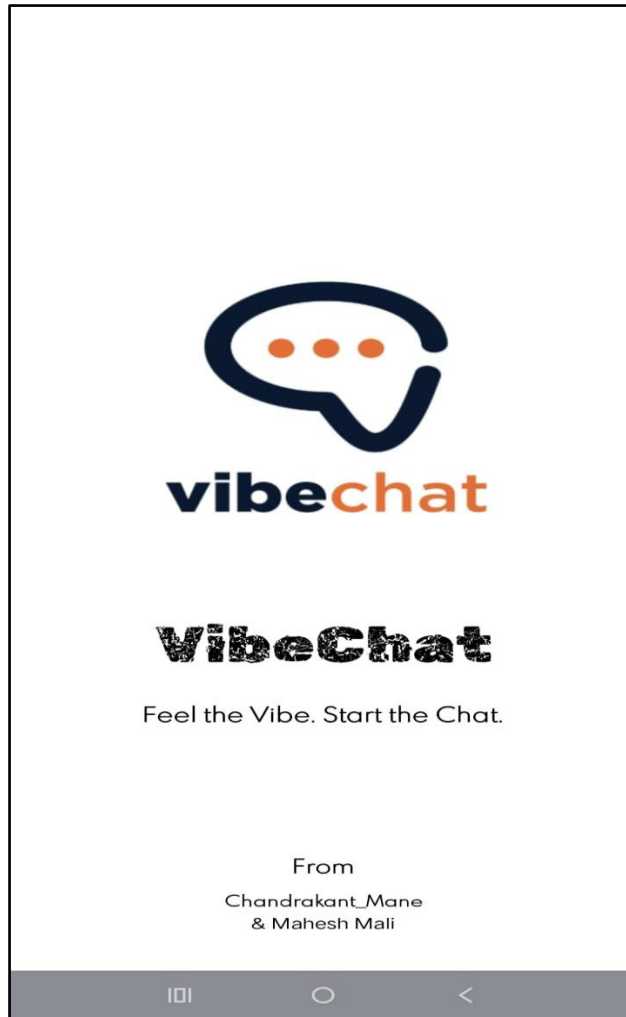
Android Application for Chatting

6.Deleted Message Table -

Field Name	Data Type	Description
Delete Id (PK)	String	Unique delete record ID
Message Id (FK)	String	Deleted message ID
Deleted By User Id (FK)	String	User who deleted
Delete Time	Long	Deletion timestamp

4.2 Input Design –

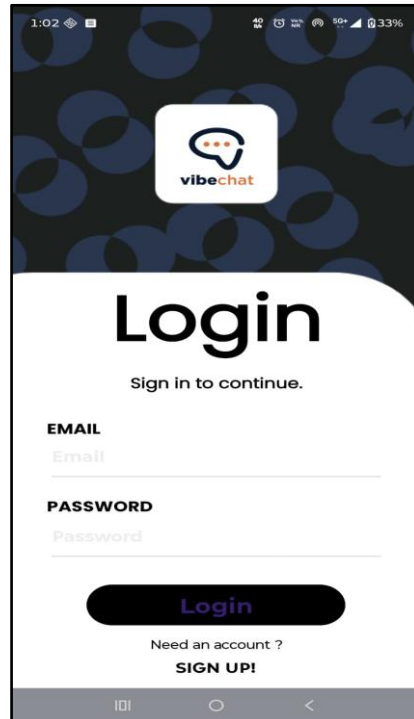
1. Splash Screen –



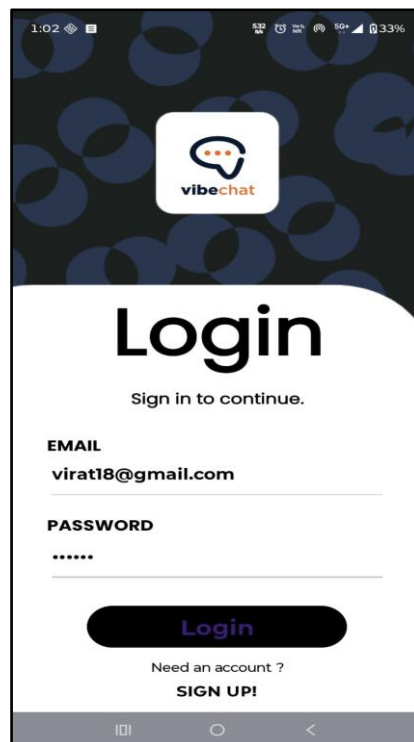
Android Application for Chatting

2. Login Page

Without Data-



With Data



Android Application for Chatting

3. Registration Page-

Without Data-



A screenshot of an Android application's registration page titled "Create New Account". The page features a dark blue header with the title in white. Below the header is a white rounded rectangle containing a camera icon in a black circle. Underneath the icon are six input fields with light blue placeholder text: "UserName", "Email", "Select Birthdate", "Select Gender", "Password", and "Confirm Password". At the bottom of the white area is a dark blue rounded button with the text "SIGN-UP" in white, and below that, the text "Login!". The status bar at the top shows the time as 1:02, signal strength, 5G+, and battery level at 33%. The bottom navigation bar shows three icons: a home icon, a circle, and a back arrow.

With Data

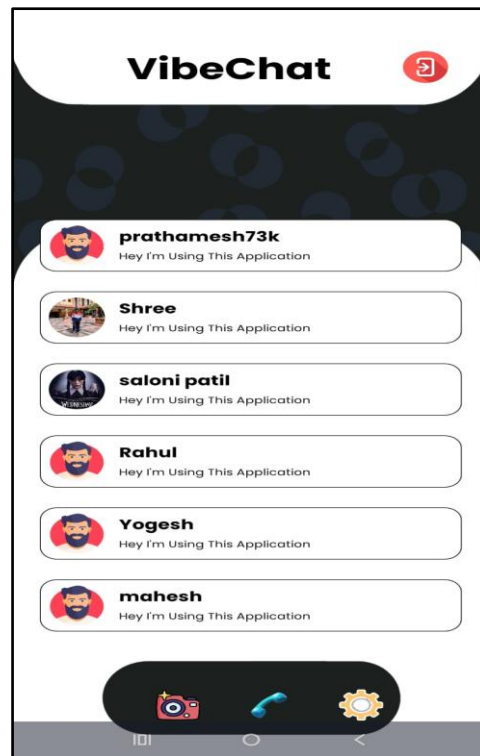


A screenshot of the same Android application's registration page, but now filled with data. The header and camera icon remain the same. The input fields now contain the following text: "Mahesh Mali" for Username, "mahesh12@gmail.com" for Email, "31/3/2003" for Birthdate, "Male" for Gender, and two rows of "*****" for Password and Confirm Password. The "SIGN-UP" button and "Login!" text are still present at the bottom. The status bar at the top shows the time as 1:04, signal strength, 5G+, and battery level at 32%. The bottom navigation bar shows the same three icons as the previous screenshot.

Android Application for Chatting

4. Chat Window-

The Main Dashboard serves as the central hub of the application, displaying a real-time list of registered users fetched from Firebase for instant communication. It features a modern, intuitive interface with a custom floating navigation bar that provides quick access to the camera, video calling, and account settings.



Source code-

MainActivity.java-

```
auth = FirebaseAuth.getInstance();
database = FirebaseDatabase.getInstance();

if (auth.getCurrentUser() == null) {
    startActivity(new Intent(MainActivity.this, Login.class));
    finish();
}

// Permission Handling
private void checkAllPermissions() {
```

Android Application for Chatting

```
List<String> list = new ArrayList<>();

    if (ContextCompat.checkSelfPermission(this,
Manifest.permission.CAMERA)

        != PackageManager.PERMISSION_GRANTED)

        list.add(Manifest.permission.CAMERA);

    if (ContextCompat.checkSelfPermission(this,
Manifest.permission.RECORD_AUDIO)

        != PackageManager.PERMISSION_GRANTED)

        list.add(Manifest.permission.RECORD_AUDIO);

    if (Build.VERSION.SDK_INT >= Build.VERSION_CODES.TIRAMISU)
    {

        if (ContextCompat.checkSelfPermission(this,
Manifest.permission.POST_NOTIFICATIONS)

            != PackageManager.PERMISSION_GRANTED)

            list.add(Manifest.permission.POST_NOTIFICATIONS);

    }

    if (!list.isEmpty())

        ActivityCompat.requestPermissions(this,
list.toArray(new String[0]), 100);
}

// Fetch Users
database.getReference().child("user").addValueEventListener(new ValueEventListener() {

    @Override

    public void onDataChange(@NonNull DataSnapshot snapshot) {

        usersArrayList.clear();

        String currentUid = auth.getUid();

        for (DataSnapshot ds : snapshot.getChildren()) {
```

Android Application for Chatting

```
        Users user = ds.getValue(Users.class);
        if (user != null &&
!user.getUserId().equals(currentUid)) {
            usersArrayList.add(user);
        }
    }
    adapter.notifyDataSetChanged();
}
@Override
public void onCancelled(@NonNull DatabaseError error) {}
});

// User Status
@Override
protected void onResume() {
    super.onResume();
    updateStatus("Online");
    sessionStartTime = System.currentTimeMillis();
}

@Override
protected void onPause() {
    super.onPause();
    long endTime = System.currentTimeMillis();
    updateStatus(String.valueOf(endTime));
    long duration = endTime - sessionStartTime;
    DatabaseReference ref = database.getReference()
        .child("user")
        .child(auth.getUid())
        .child("totalActiveTime");
```

Android Application for Chatting

```
        ref.addListenerForSingleValueEvent(new
ValueEventListener() {

            @Override

            public void onDataChange(@NonNull DataSnapshot
snapshot) {

                long total = snapshot.exists() ?
snapshot.getValue(Long.class) : 0;

                ref.setValue(total + duration);

            }

            @Override public void onCancelled(@NonNull
DatabaseError error) {}

        });
    }

    private void updateStatus(String status) {

        database.getReference().child("user")

            .child(auth.getUid())

            .child("userStatus")

            .setValue(status);

    }

    // Device Token

    FirebaseMessaging.getInstance().getToken().addOnSuccessListene
r(token -> {

        database.getReference().child("user")

            .child(auth.getUid())

            .child("token")

            .setValue(token);

    });

    // Camera

    private void openCamera() {

        ContentValues values = new ContentValues();

        values.put(MediaStore.Images.Media.TITLE, "New Picture");
```

Android Application for Chatting

```
        imageUrl = getResolver().insert(
            MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
            values);

        Intent intent = new
        Intent(MediaStore.ACTION_IMAGE_CAPTURE);

        intent.putExtra(MediaStore.EXTRA_OUTPUT, imageUrl);
        startActivityForResult(intent, 10);
    }

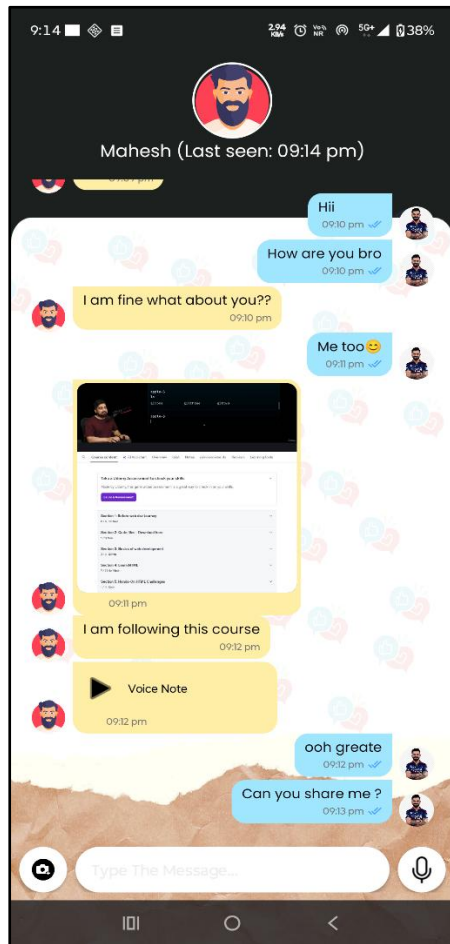
    // Logout
    auth.signOut();
    startActivity(new Intent(MainActivity.this, Login.class));
    finish();

    // On Disconnect
    database.getReference().child("user")
        .child(auth.getUid())
        .child("userStatus")
        .onDisconnect()
        .setValue(String.valueOf(System.currentTimeMillis()));
```

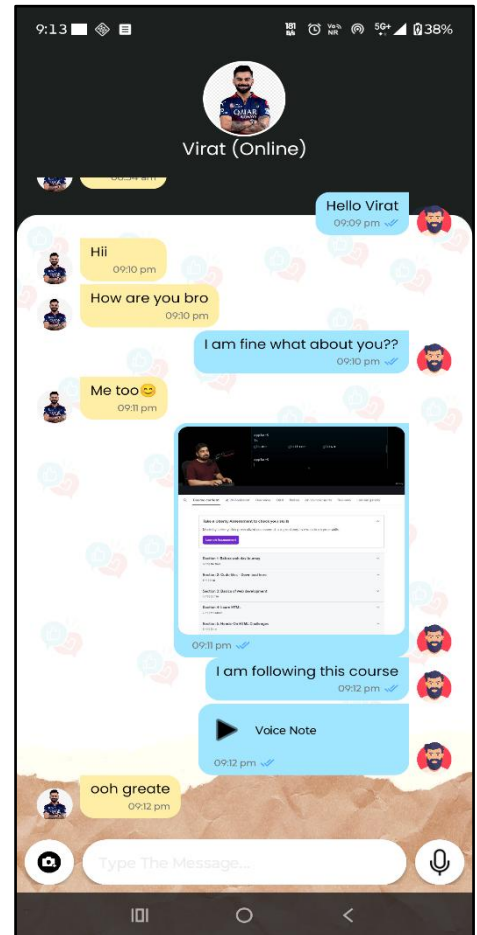
Android Application for Chatting

5. Chat Window

User 1-



User 2-



Source Code-

chatWindow.java-

```
database = FirebaseDatabase.getInstance();  
firebaseAuth = FirebaseAuth.getInstance();  
storage = FirebaseStorage.getInstance();  
reciverName = getIntent().getStringExtra("namee");  
reciverimg = getIntent().getStringExtra("reciverImg");  
reciverUid = getIntent().getStringExtra("uid");  
SenderUID = firebaseAuth.getUid();
```

Android Application for Chatting

```
senderRoom = SenderUID + receiverUid;
receiverRoom = receiverUid + SenderUID;

audioPath = getExternalCacheDir().getAbsolutePath() +
"/voice_note.3gp";

// Fetch sender profile image
database.getReference().child("user").child(SenderUID).child("
profilepic")
.addListenerForSingleValueEvent(new ValueEventListener() {
    @Override
    public void onDataChange(@NonNull DataSnapshot snapshot) {
        if (snapshot.exists()) {
            senderImg = snapshot.getValue().toString();
        }
    }
    @Override public void onCancelled(@NonNull DatabaseError
error) {}
});

// Typing Status
textmsg.addTextChangedListener(new TextWatcher() {
    @Override public void beforeTextChanged(CharSequence s,
int start, int count, int after) {}

    @Override
    public void onTextChanged(CharSequence s, int start, int
before, int count) {
        if (s.toString().trim().length() > 0) {

database.getReference().child("user").child(SenderUID)

.child("typingStatus").setValue(receiverUid);
        } else {
```

Android Application for Chatting

```
database.getReference().child("user").child(SenderUID)
    .child("typingStatus").setValue("false");
    }
}

@Override public void afterTextChanged(Editable s) {}
});

// Online / Last Seen + Typing
database.getReference().child("user").child(reciverUid)
.addValueEventListener(new ValueEventListener() {
    @Override
    public void onDataChange(@NonNull DataSnapshot snapshot) {
        String status =
snapshot.child("userStatus").getValue(String.class);
        String typing =
snapshot.child("typingStatus").getValue(String.class);

        if (typing != null && typing.equals(SenderUID)) {
            reciverNName.setText(reciverName + "
(typing...)");
        } else if (status != null) {
            if (status.equals("Online")) {
                reciverNName.setText(reciverName + "
(Online)");
            } else {
                try {
                    long time = Long.parseLong(status);
                    java.text.SimpleDateFormat sdf =
                        new java.text.SimpleDateFormat("hh:mm
a", Locale.getDefault());
                    reciverNName.setText(reciverName + " (Last
seen: " + sdf.format(new Date(time)) + ")");
                } catch (Exception e) {
```

Android Application for Chatting

```
                receiverNName.setText(receiverName + "
(Offline)");
            }
        }
    }
}

@Override public void onCancelled(@NonNull DatabaseError
error) {}
});

// Load Messages + Seen Logic

chatRef =
database.getReference().child("chats").child(senderRoom).child
("messages");

chatListener = chatRef.addValueEventListener(new
 ValueEventListener() {
    @Override
    public void onDataChange(@NonNull DataSnapshot snapshot) {
        messagessArrayList.clear();

        for (DataSnapshot ds : snapshot.getChildren()) {
            MsgModelclass msg =
ds.getValue(MsgModelclass.class);

            if (msg != null) {
                msg.setMsgId(ds.getKey());
                messagessArrayList.add(msg);

                if (!msg.getSenderId().equals(SenderUID) &&
!msg.isSeen()) {
                    HashMap<String, Object> map = new
HashMap<>();
                    map.put("seen", true);
```

Android Application for Chatting

```
        map.put("seenTimeStamp",
System.currentTimeMillis());

        ds.getRef().updateChildren(map);
        database.getReference().child(reciverRoom)

.child("messages").child(ds.getKey()).updateChildren(map);
    }
}
}
mmessagesAdppter.notifyDataSetChanged();
}

@Override public void onCancelled(@NonNull DatabaseError
error) {}
});

// Send Button Logic
sendbtn.setOnClickListener(view -> {
    String message = textmsg.getText().toString().trim();
    if (message.length() > 0 || selectedImageUri != null) {
        if (selectedImageUri != null)
            uploadImageWithCaption(selectedImageUri, message);
        else
            sendTextMessage(message);
    } else {
        if (checkAudioPermission()) startVoiceRecordingUI();
    }
});

// Send Text Message
private void sendTextMessage(String message) {
    String key = database.getReference().push().getKey();
```

Android Application for Chatting

```
MsgModelclass msg = new MsgModelclass(
    SenderUID,
    System.currentTimeMillis(),
    message,
    "text",
    key,
    null
);

database.getReference().child("chats").child(senderRoom)
    .child("messages").child(key).setValue(msg);

database.getReference().child("chats").child(reciverRoom)
    .child("messages").child(key).setValue(msg);

textmsg.setText("");
}

// Upload Image Message
private void uploadImageWithCaption(Uri uri, String caption) {
    String key = database.getReference().push().getKey();
    StorageReference ref =
storage.getReference().child("chats").child(key);

    ref.putFile(uri).addOnCompleteListener(task -> {
        if (task.isSuccessful()) {

ref.getDownloadUrl().addOnSuccessListener(downloadUri -> {

                MsgModelclass msg = new MsgModelclass(
                    SenderUID,
```

Android Application for Chatting

```
        System.currentTimeMillis(),
        caption,
        "image",
        key,
        downloadUri.toString()
    );

    database.getReference().child("chats").child(senderRoom)

    .child("messages").child(key).setValue(msg);

    database.getReference().child("chats").child(reciverRoom)

    .child("messages").child(key).setValue(msg);
    });
}

});
}

// Voice Recording
private void startRecording() {
    recorder = new MediaRecorder();
    recorder.setAudioSource(MediaRecorder.AudioSource.MIC);

    recorder.setOutputFormat(MediaRecorder.OutputFormat.THREE_GPP);
    ;

    recorder.setOutputFile(audioPath);

    recorder.setAudioEncoder(MediaRecorder.AudioEncoder.AMR_NB);

    try {
        recorder.prepare();
        recorder.start();
        isRecording = true;
    }
```

Android Application for Chatting

```
        } catch (IOException e) {
            e.printStackTrace();
        }
    }

private void stopRecording() {
    if (isRecording) {
        recorder.stop();
        recorder.release();
        recorder = null;
        isRecording = false;
    }
}

// Upload Voice Message
private void uploadAudioFile() {
    StorageReference ref = FirebaseStorage.getInstance()
        .getReference().child("voice")
        .child(database.getReference().push().getKey());

    ref.putFile(Uri.fromFile(new File(audioPath)))
        .addOnCompleteListener(task -> {
            if (task.isSuccessful()) {
                ref.getDownloadUrl().addOnSuccessListener(uri -> {
                    String key =
database.getReference().push().getKey();
                    MsgModelclass msg = new MsgModelclass(
                        SenderUID,
                        System.currentTimeMillis(),
                        "",
                        "audio",
```

Android Application for Chatting

```
        key,
        uri.toString()
    );

database.getReference().child("chats").child(senderRoom)

.child("messages").child(key).setValue(msg);

database.getReference().child("chats").child(reciverRoom)

.child("messages").child(key).setValue(msg);
    });
}
});
}

// Camera & Gallery
private void openCamera() {
    ContentValues values = new ContentValues();
    values.put(MediaStore.Images.Media.TITLE, "New Picture");

    selectedImageUri = getContentResolver().insert(
        MediaStore.Images.Media.EXTERNAL_CONTENT_URI,
values);

    Intent intent = new
Intent(MediaStore.ACTION_IMAGE_CAPTURE);

    intent.putExtra(MediaStore.EXTRA_OUTPUT,
selectedImageUri);

    startActivityResult(intent, 10);
}
```

Android Application for Chatting

```
private void openGallery() {
    Intent intent = new Intent(Intent.ACTION_PICK,
        MediaStore.Images.Media.EXTERNAL_CONTENT_URI);
    startActivityForResult(intent, 25);
}

// User Status
@Override
protected void onResume() {
    super.onResume();
    if (SenderUID != null) {
        database.getReference().child("user")

.child(SenderUID).child("userStatus").setValue("Online");
    }
}

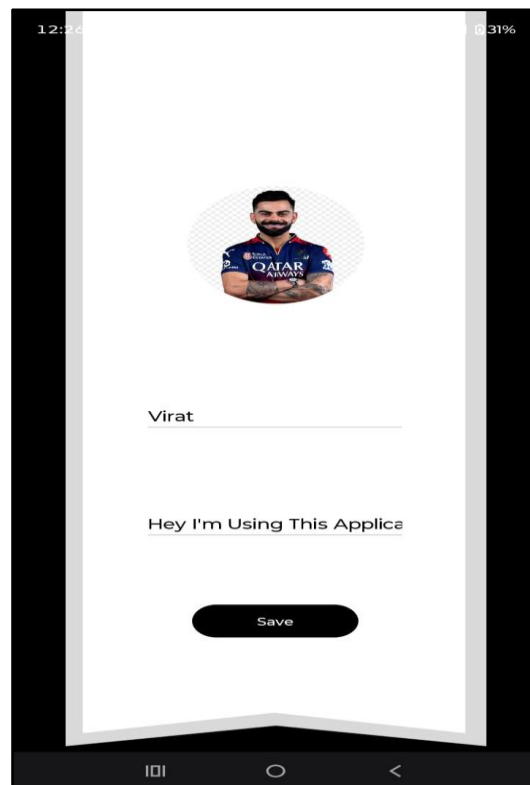
@Override
protected void onPause() {
    super.onPause();
    if (SenderUID != null) {
        database.getReference().child("user")
            .child(SenderUID)
            .child("userStatus")

.setValue(String.valueOf(System.currentTimeMillis()));
        database.getReference().child("user")
            .child(SenderUID)
            .child("typingStatus")
            .setValue("false");
    }
    if (chatRef != null && chatListener != null) {
```

Android Application for Chatting

```
        chatRef.removeEventListener(chatListener);  
    }  
}
```

6. Profile update section-



Source code-

Setting.java-

```
auth = FirebaseAuth.getInstance();  
database = FirebaseDatabase.getInstance();  
storage = FirebaseStorage.getInstance();  
  
// Fetch existing user data  
DatabaseReference reference =  
    database.getReference().child("user").child(auth.getUid());  
  
reference.addListenerForSingleValueEvent(new  
    ValueEventListener() {
```

Android Application for Chatting

```
@Override

public void onDataChange(@NonNull DataSnapshot snapshot) {

    if (snapshot.exists()) {

        email =
snapshot.child("mail").getValue().toString();

        password =
snapshot.child("password").getValue().toString();

        existingProfilePic =
snapshot.child("profilepic").getValue().toString();


        String name =
snapshot.child("userName").getValue().toString();

        String status =
snapshot.child("status").getValue().toString();


        birthdate = snapshot.hasChild("birthdate") ?
snapshot.child("birthdate").getValue().toString() : "";

        gender = snapshot.hasChild("gender") ?
snapshot.child("gender").getValue().toString() : "";

        token = snapshot.hasChild("token") ?
snapshot.child("token").getValue().toString() : "";

        setname.setText(name);

        setstatus.setText(status);

    }

}

@Override public void onCancelled(@NonNull DatabaseError
error) {}

});

// Select profile image
setprofile.setOnClickListener(view -> {

    Intent intent = new Intent();

    intent.setType("image/*");

    intent.setAction(Intent.ACTION_GET_CONTENT);
```

Android Application for Chatting

```
        startActivityForResult(Intent.createChooser(intent,
"Select Picture"), 10);
    });

// Save profile changes
donebut.setOnClickListener(v -> {
    String name = setname.getText().toString();
    String Status = setstatus.getText().toString();

    if (setImageUri != null) {
        try {
            Bitmap bitmap =
ImageUtils.handleSamplingAndRotationBitmap(setting.this,
setImageUri);

            ByteArrayOutputStream baos = new
ByteArrayOutputStream();

            bitmap.compress(Bitmap.CompressFormat.JPEG, 70,
baos);

            byte[] data = baos.toByteArray();

            StorageReference storageReference =
storage.getReference()
                .child("Upload").child(auth.getUid());

            storageReference.putBytes(data).addOnCompleteListener(task ->
            {
                if (task.isSuccessful()) {

                    storageReference.getDownloadUrl().addOnSuccessListener(uri ->
                    {
                        saveUserToDatabase(name, Status,
uri.toString());
                    });
                }
            })
        }
    }
});
```

Android Application for Chatting

```
        });

        } catch (Exception e) {
            e.printStackTrace();
        }
    } else {
        saveUserToDatabase(name, Status, existingProfilePic);
    }
});

// Save user data to Firebase
private void saveUserToDatabase(String name, String status,
String profileUrl) {

    Users users = new Users(
        auth.getUid(),
        name,
        email,
        password,
        profileUrl,
        status,
        birthdate,
        gender,
        "Online"
    );

    users.setToken(token);
    users.setTypingStatus("false");

    database.getReference().child("user")
        .child(auth.getUid())
        .setValue(users);
}
```

Android Application for Chatting

```
}

// User status handling
private void updateStatus(String status) {
    if (auth.getUid() != null) {
        database.getReference().child("user")
            .child(auth.getUid())
            .child("userStatus")
            .setValue(status);
    }
}

@Override
protected void onResume() {
    super.onResume();
    updateStatus("Online");
}

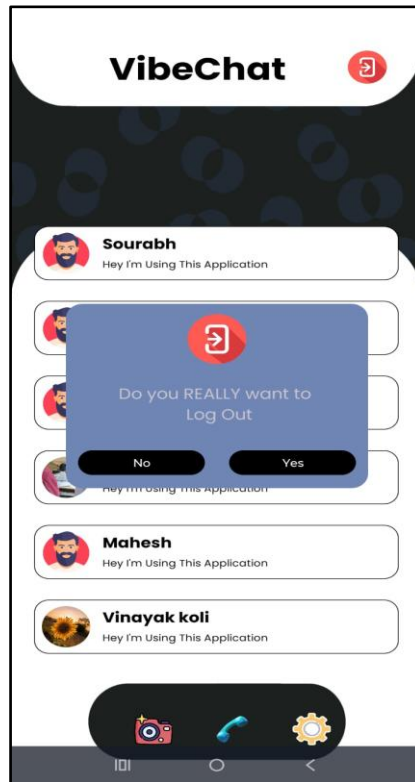
@Override
protected void onPause() {
    super.onPause();
    updateStatus(String.valueOf(System.currentTimeMillis()));
}

// Image selection result
@Override
protected void onActivityResult(int requestCode, int
resultCode, @Nullable Intent data) {
    if (requestCode == 10 && resultCode == RESULT_OK && data
!= null) {
        setImageUri = data.getData();
        setprofile.setImageURI(setImageUri);
    }
}
```

Android Application for Chatting

}

7.Account logout-



Android Application for Chatting

4.3 Output Design –

1. User Registration Report

12:22 VibeChat Admin Reports 5G+ 31%				
User Reg	Male	Female	Age	Daily
userId	Name	Email	Dob	
98pxW4bkFMXhEea6bU LP5hoEjW23	prathamesh73k	prathamesh73k@gmail .com	7/3/2006	
BPyL5S0h2IOd2MQf9MZ uS6ugDQI2	Shree	shreegaming366@gmail .com	18/7/2005	
PjPOw2tHYOgOgqsfwjP Ud088CDi2	saloni patil	salonipatil@gmail.com	1/10/2005	
U5gXNjeNpBhVnQyzuAX NfQFiP863	Virat	virat18@gmail.com	5/11/1988	
bI3P6Pg9ZTdqa6JCw08 WThEE6G13	Rahul	rahul18@gmail.com	5/3/2005	
fNvjvV68PFdugZHb6Yw ozrNq4yO2	Yogesh	yogi@gmail.com	18/7/2005	
jQWdJ0yb9TZOhFC9Bb2 KuBHY7lu2	Sourabh	sourabh@gmail.com	17/3/1996	
jtbeeGQFRfhYxHaDGjUG KpJMDzr1	mahesh	malimahesh123@gmail .com	31/3/2003	
nGT8wo90PcNhlyrprjr5z WCTD7v2	Mane23	mane2610@gmail.com	18/4/2004	
p2LeFHp93PPzgmzn4EV 4Uz6aH6V2	Swapnali73k	swapnali@gmail.com	7/3/2000	
qbsNPYrzVdbUe4HNEAz cpBgDDcC3	Mahesh	mahesh123@gmail.com	21/2/2026	
sUlXhBDPqZS2ucF3bCH VgZqOZfA3	Vinayak koli	koliv0342@gmail.com	21/3/2003	

Android Application for Chatting

2. Male user report

12:22  393 						
User Reg	Male	Female	Age	Daily		
Name	Gen	Dob	Email	Stat	Reg	Log
prathamesh73k	Male	7/3/2006	prathamesh73k@gmail.com	Hey I'm Using This Application	15-03-2026	15-03-2026
Shree	Male	18/7/2005	shreegaminig366@gmail.com	Hey I'm Using This Application	12-03-2026	18-03-2026
Virat	Male	5/11/1988	virat18@gmail.com	Hey I'm Using This Application	18-03-2026	18-03-2026
Rahul	Male	5/3/2005	rahul18@gmail.com	Hey I'm Using This Application	05-03-2026	17-03-2026
Yogesh	Male	18/7/2005	yogi@gmail.com	Hey I'm Using This Application	07-03-2026	07-03-2026
Sourabh	Male	17/3/1996	sourabh@gmail.com	Hey I'm Using This Application	17-03-2026	18-03-2026
mahesh	Male	31/3/2003	malimahesh123@gmail.com	Hey I'm Using This Application	05-03-2026	18-03-2026
Mane23	Male	18/4/2004	mane2610@gmail.com	Hey I'm Using This Application	N/A	N/A
Mahesh	Male	21/2/2026	mahesh123@gmail.com	Hey I'm Using This Application	N/A	22-02-2026
Vinayak koli	Male	21/3/2003	koliv0342@gmail.com	Hey I'm Using This Application	17-03-2026	18-03-2026

3. Female user report-

12:22  912 						
User Reg	Male	Female	Age	Daily		
Name	Gen	Dob	Email	Stat	Reg	Log
saloni patil	Female	1/10/2005	salonipatil@gmail.com	Hey I'm Using This Application	07-03-2026	11-03-2026
Swapnali73k	Female	7/3/2000	swapnali@gmail.com	Hey I'm Using This Application	07-03-2026	11-03-2026

Android Application for Chatting

4. Age Wise user report

The screenshot shows the 'VibeChat Admin Reports' app interface. At the top, there's a status bar with time 12:23, battery 31%, and 5G+ signal. Below the title bar, there are filters: 'Age' (selected), 'Male', 'Female', and 'Daily'. A search bar contains the number '21' and a 'Show' button. Below this is a table with columns: Name, Dob, Age, Gen, Email, and Active.

Name	Dob	Age	Gen	Email	Active
Shree	18/7/2005	21	Male	shreegamin g366@gmail .com	0h 4m
saloni patil	1/10/2005	21	Female	salonipatil @gmail .com	0h 2m
Rahul	5/3/2005	21	Male	rahul18@gmail .com	0h 1m
Yogesh	18/7/2005	21	Male	yogi@gmail .com	0h 0m

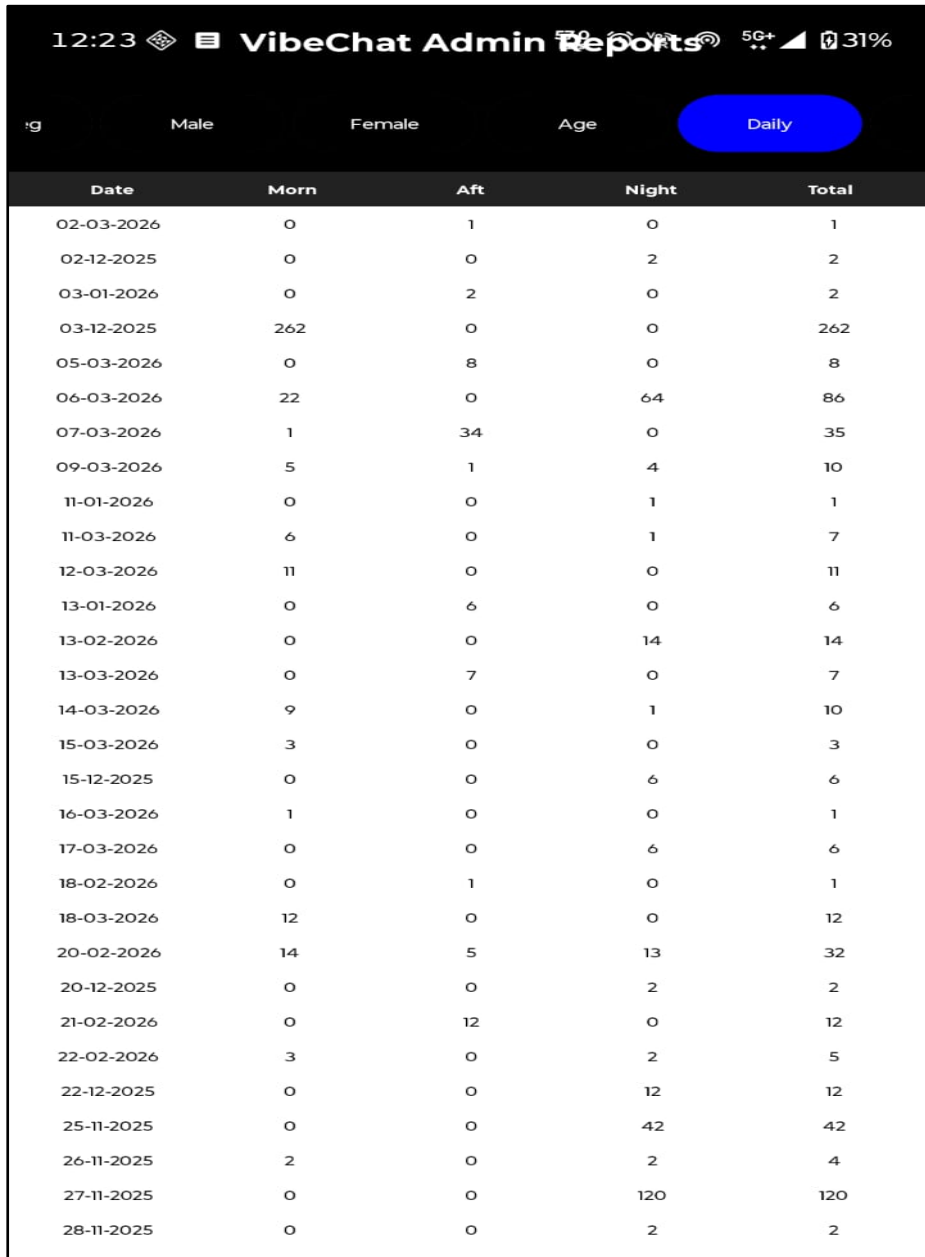
5. User Activity report-

The screenshot shows the 'VibeChat Admin Reports' app interface. At the top, there's a status bar with time 12:23, battery 31%, and 5G+ signal. Below the title bar, there are filters: 'email', 'Age', 'Daily', 'Stats', and 'Activity' (selected). Below this is a table with columns: Name, Morn, Aft, Night, and Total.

Name	Morn	Aft	Night	Total
Vinayak koli	7	8	30	45
Rahul	0	1	1	2
Swapnali73k	0	4	1	5
Mahesh	2	0	1	3
Shree	11	2	0	13
Virat	36	31	45	112
saloni patil	0	5	0	5
prathamesh73k	2	0	0	2
Mane23	0	7	9	16
mahesh	15	4	0	19

Android Application for Chatting

6. Daily Usage report

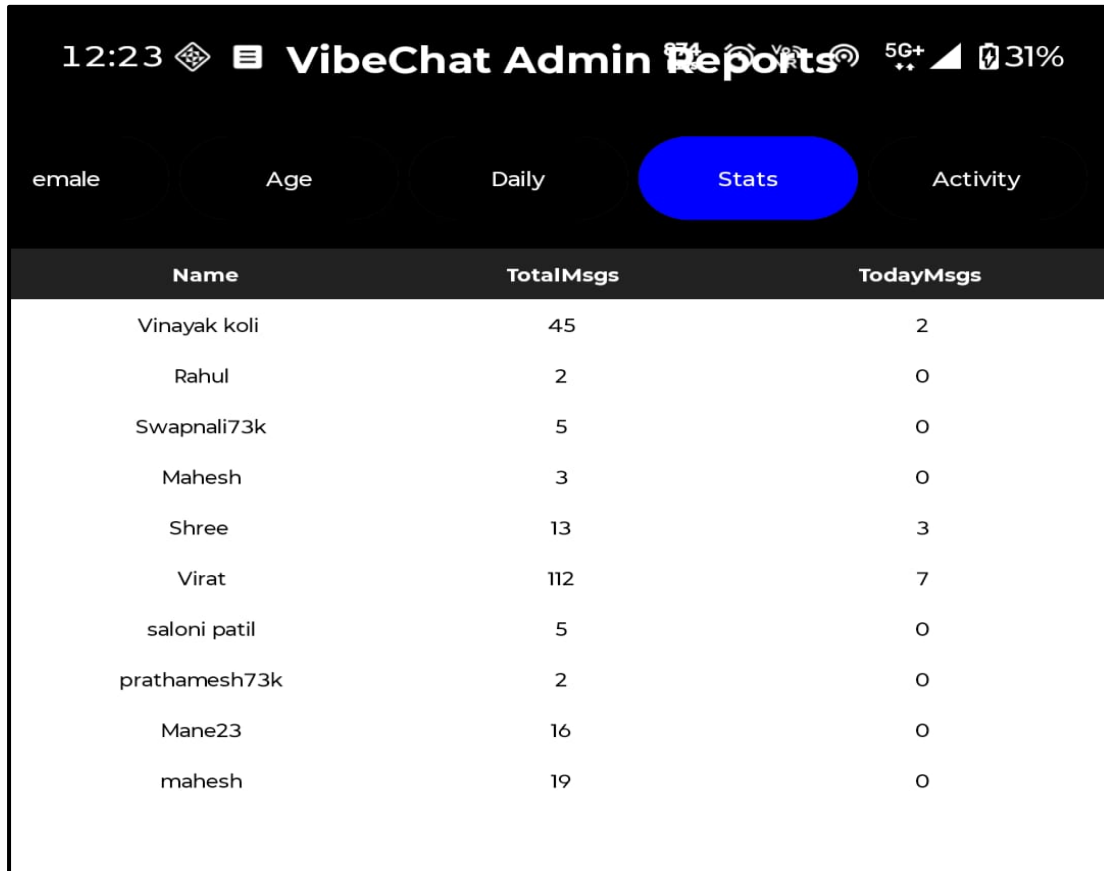


The screenshot displays the 'VibeChat Admin Reports' application interface. At the top, the status bar shows the time as 12:23, signal strength, 5G+ connectivity, and a 31% battery level. Below the title bar, there are filter buttons for 'Male', 'Female', 'Age', and a selected 'Daily' button. The main content area features a table with the following columns: 'Date', 'Morn', 'Aft', 'Night', and 'Total'. The table lists usage data for various dates from 02-03-2026 to 28-11-2025.

Date	Morn	Aft	Night	Total
02-03-2026	0	1	0	1
02-12-2025	0	0	2	2
03-01-2026	0	2	0	2
03-12-2025	262	0	0	262
05-03-2026	0	8	0	8
06-03-2026	22	0	64	86
07-03-2026	1	34	0	35
09-03-2026	5	1	4	10
11-01-2026	0	0	1	1
11-03-2026	6	0	1	7
12-03-2026	11	0	0	11
13-01-2026	0	6	0	6
13-02-2026	0	0	14	14
13-03-2026	0	7	0	7
14-03-2026	9	0	1	10
15-03-2026	3	0	0	3
15-12-2025	0	0	6	6
16-03-2026	1	0	0	1
17-03-2026	0	0	6	6
18-02-2026	0	1	0	1
18-03-2026	12	0	0	12
20-02-2026	14	5	13	32
20-12-2025	0	0	2	2
21-02-2026	0	12	0	12
22-02-2026	3	0	2	5
22-12-2025	0	0	12	12
25-11-2025	0	0	42	42
26-11-2025	2	0	2	4
27-11-2025	0	0	120	120
28-11-2025	0	0	2	2

Android Application for Chatting

7. Chat Stats report



The screenshot displays the 'VibeChat Admin Reports' app interface. At the top, the status bar shows the time 12:23, signal strength, 5G+ connectivity, and 31% battery. The app title 'VibeChat Admin Reports' is centered. Below the title, there are five filter buttons: 'emale', 'Age', 'Daily', 'Stats' (highlighted in blue), and 'Activity'. The main content area features a table with three columns: 'Name', 'TotalMsgs', and 'TodayMsgs'. The table lists ten users with their respective message counts.

Name	TotalMsgs	TodayMsgs
Vinayak koli	45	2
Rahul	2	0
Swapnali73k	5	0
Mahesh	3	0
Shree	13	3
Virat	112	7
saloni patil	5	0
prathamesh73k	2	0
Mane23	16	0
mahesh	19	0

Chapter 5

System Implementation

5.1 User Manual

System Implementation

User Manual

The User Manual provides step-by-step instructions for users to operate the Vibe Chat Application effectively. It explains how a user can install the application, create an account, log in, send messages, and use other features available in the system. The manual helps users understand the working of the application in a simple and user-friendly manner.

1 Installation of the Application

To use the Vibe Chat Application, the user must first install the application on an Android device. The installation process is simple and requires the following steps:

1. Download the Vibe Chat Application APK file from the provided source.
2. Open the downloaded file on the Android device.
3. Allow installation from unknown sources if required.
4. Tap on the Install button.
5. After installation is complete, the application icon will appear on the device screen.
6. Tap the icon to launch the application.

2 Starting the Application

When the user opens the application, the Splash Screen is displayed.

This screen shows the application logo and loads the required resources for the app. After a few seconds, the user is redirected to the Login or Registration screen.

3 User Registration (Sign Up)

New users must create an account before using the application.

Steps for Registration:

1. Open the application.
2. Click on the Sign Up / Register option.

Android Application for Chatting

3. Enter the required details such as:
 - Username
 - Email ID or Mobile Number
 - Password
4. Click on the Register button.
5. The system validates the details and stores them in the database.
6. After successful registration, the user can log in to the application.

4 User Login

Registered users can access the system by logging in.

Steps for Login:

1. Open the Vibe Chat Application.
2. Enter the registered email/mobile number.
3. Enter the password.
4. Click on the Login button.
5. If the entered details are correct, the system allows the user to access the chat interface.

5 Profile Management

Users can manage and update their profile information.

Features of Profile Management:

- View personal details
- Update username or profile picture
- Update status message

Android Application for Chatting

Steps:

1. Go to the Profile Section.
2. Click on Edit Profile.
3. Update the required information.
4. Click Save to update the changes.

6 Chat / Messaging Feature

The main feature of the Vibe Chat Application is real-time messaging between users.

Steps to send a message:

1. Open the application and log in.
2. Select a contact from the contact list.
3. Type the message in the message box.
4. Click the Send button.
5. The message is delivered to the receiver instantly.

If the receiver is offline, the message is stored in the database and delivered when the receiver logs in.

7 Voice Message Feature

The application also allows users to send voice messages.

Steps:

1. Open the chat with the selected contact.
2. Press and hold the Voice Message / Microphone icon.
3. Record the voice message.
4. Release the button to send the recorded message.
5. The receiver can listen to the voice message directly in the chat.

This feature allows quick communication without typing messages.

Android Application for Chatting

8 Media Sharing

The application also allows users to send media files such as photos or videos.

Steps:

1. Open the chat with the selected contact.
2. Click the Attachment / Camera icon.
3. Choose media from the gallery or capture a photo using the camera.
4. Click Send.
5. The media file is uploaded and shared with the receiver.

9 Video Calling Feature

Users can also make video calls using the application.

Steps:

1. Open the chat of the selected contact.
2. Click on the Video Call Icon.
3. The system connects both users using an internet connection.
4. Once the receiver accepts the call, the video call starts.

10 Settings

The Settings section allows users to manage application preferences and account settings.

Features of Settings:

- Change password
- Manage notification settings
- Update privacy settings
- Update profile details

Android Application for Chatting

Steps:

1. Open the Menu or Settings option.
2. Select the desired setting option.
3. Make the required changes.
4. Click Save to apply the changes.

11 Viewing Chat History

The system automatically stores all messages in the database.

Users can:

- View previous messages
- Scroll through old conversations
- Access chat history anytime

This helps users keep track of their communication records.

12 Logout

To ensure security, users can log out of the application.

Steps:

1. Open the Menu or Settings option.
2. Click on Logout.
3. The user session ends and the system redirects to the login screen.

Chapter 6

System Maintenance

Android Application for Chatting

6.1 Conclusion

The Vibe Chat Android Application was successfully designed and developed to provide a simple, secure, and efficient real-time communication platform for users. The main objective of this project was to create an Android-based messaging application that allows users to communicate instantly through text messages and share media files using an internet connection.

The system was developed using modern technologies such as Java for application logic and XML for user interface design, which helped in creating a responsive and user-friendly application. The application provides essential features such as user registration, login authentication, real-time messaging, media sharing, profile management, and chat history storage. These features make communication easier and more convenient for users.

During the development of this project, important concepts such as Android application development, database management, client-server communication, and user authentication were implemented. This project also helped in gaining practical knowledge about designing and developing mobile applications.

Overall, the Vibe Chat Application provides a reliable communication platform and fulfills the objectives of real-time messaging. The system is easy to use, efficient, and capable of supporting further improvements in the future.

6.2 Further Enhancement

The Vibe Chat Android Application developed in this project provides basic real-time communication features such as user registration, login, messaging, media sharing, and video calling. Although the system works efficiently, there are several possibilities for future improvements to make the application more advanced and user-friendly.

Some of the possible future enhancements are as follows:

1. Group Chat Feature

Currently, the application mainly supports one-to-one communication. In future versions, a group chat feature can be added so that multiple users can communicate in a single conversation.

2. Voice Calling Feature

In addition to video calling, voice calling functionality can be added to allow users to make audio calls through the internet.

4. File and Document Sharing

The application can be enhanced to allow users to share different types of files such as PDF documents, presentations, and other attachments through the chat interface.

5. End-to-End Encryption

To improve security and privacy, end-to-end encryption can be implemented so that messages can only be read by the sender and the receiver.

7. Cloud Backup and Data Recovery

Cloud backup functionality can be added to allow users to store and restore their chat history when changing devices.

8. Multi-Platform Support

The application can be extended to support other platforms such as Web and iOS, allowing users to access their chats from multiple devices.

Overall, these enhancements will improve the functionality, performance, and security of the Vibe Chat Application and make it more comparable to modern messaging applications.

Chapter 7

Bibliography

7.1 Bibliography –

- Android Developers Documentation. (n.d.). *Activity Lifecycle, Intent, and UI Layouts*. Retrieved from: <https://developer.android.com/docs>
- Firebase Documentation. (n.d.). *Firebase Realtime Database, Authentication, and Cloud Storage for Android*. Google. Retrieved from: <https://firebase.google.com/docs>
- Firebase Cloud Messaging (FCM). (n.d.). *Sending downstream messages via HTTP/REST*. Google. Retrieved from: <https://firebase.google.com/docs/cloud-messaging>
- ZegoCloud Documentation. (n.d.). *Zego UIKit Prebuilt Call Invitation SDK for Android*. Retrieved from: <https://docs.zegocloud.com/>

2. Third-Party Libraries & Open Source Tools

- Dodenhof, H. (2021). *CircleImageView: A fast circular ImageView for Android*. [GitHub Repository]. Retrieved from: <https://github.com/hdodenhof/CircleImageView>
- Intuit. (2020). *SDP - a scalable size unit & SSP - a scalable size unit for texts*. [GitHub Repository].

3. Software & IDEs

- Android Studio. (2024). *Version Iguana/Jellyfish - The Official IDE for Android Development*. Google.
- Firebase Console. (2024). *Cloud-based Backend-as-a-Service*. Google.
- Node.js & Firebase CLI. (2024). *Server-side environment for deploying Cloud Functions*.